

XEarthLayer – CPU-relevante Konfigurationseinstellungen

XEarthLayer Projektanalyse

2026-02-13

CPU-relevante Einstellungen

Direkt CPU-relevante Einstellungen

Sektion	Einstellung	Typ	Standard	Beschreibung	CPU-Auswirkung
[generation]	threads	integer	num_cpus	Anzahl Worker-Threads für parallele Tile-Generierung	Direkt: Bestimmt wie viele Tiles gleichzeitig generiert werden. Nicht höher als CPU-Kernanzahl setzen.
[executor]	cpu_concurrent	integer	num_cpus × 1.25	Gleichzeitige CPU-gebundene Operationen (Assemble + DDS-Encode)	Direkt: Begrenzt parallele BC1/BC3-Kompression und Tile-Assembly. Formel: $\max(\text{num_cpus} \times 1.25, \text{num_cpus} + 2)$

Indirekt CPU-relevante Einstellungen

Sektion	Einstellung	Typ	Standard	Beschreibung	CPU-Auswirkung
[control_plane]	max_concurrent_jobs	integer	num_cpus × 2	Maximale gleichzeitige Tile-Verarbeitungsjobs	Begrenzt Gesamtanzahl aktiver Jobs (jeder Job enthält CPU-Tasks). Bei 8 CPUs = 16 gleichzeitige Tiles.
[executor]	max_concurrent_tasks	integer	128	Maximale gleichzeitige Tasks im Executor	Obergrenze aller Tasks (Netzwerk + CPU + Disk I/O zusammen).
[executor]	network_concurrent	integer	128	Gleichzeitige HTTP-Verbindungen (64-256)	Jede Verbindung verbraucht CPU für TLS/Parsing. Formel: $\min(\text{num_cpus} \times 16, 256)$

Sektion	Einstellung	Typ	Standard	Beschreibung	CPU-Auswirkung
[cache]	disk_io_profile	string	auto	Disk-I/O-Profil (auto/hdd/ssd/nvme)	Beeinflusst I/O-Parallelität. NVMe (256 Ops) vs HDD (4 Ops) wirkt sich auf CPU-Scheduling aus.
[generation]	timeout	integer	10	Timeout pro Tile in Sekunden	Bei Timeout wird CPU-Arbeit abgebrochen. Beeinflusst wie lange CPU-Ressourcen gebunden bleiben.
[control_plane]	stall_threshold_secs	integer	60	Erkennung blockierter Jobs	Blockierte Jobs werden beendet und CPU-Ressourcen freigegeben.

Zusammenfassung: CPU-Tuning-Hierarchie

Die drei direkt CPU-relevanten Einstellungen bilden eine Hierarchie:

1. **generation.threads** — Wie viele Worker-Threads insgesamt existieren
2. **executor.cpu_concurrent** — Wie viele davon gleichzeitig CPU-intensive Arbeit (DDS-Encoding) machen dürfen
3. **control_plane.max_concurrent_jobs** — Wie viele Tiles insgesamt gleichzeitig verarbeitet werden

Beispiel: System mit 8 CPUs

Einstellung	Default-Wert	Erklärung
generation.threads	8	8 Worker-Threads (= Anzahl CPUs)
executor.cpu_concurrent	10	$\max(8 \times 1.25, 8 + 2) = 10$ parallele CPU-Ops
control_plane.max_concurrent_jobs	16	$8 \times 2 = 16$ gleichzeitige Tile-Jobs

Disk-I/O-Profil und CPU-Auswirkung

Profil	Gleichzeitige Ops	CPU-Last	Empfohlen für
hdd	1–4	Niedrig	Klassische Festplatten
ssd	32–64	Mittel	SATA/AHCI SSDs (Standard)
nvme	128–256	Hoch	NVMe-Laufwerke
auto	Automatisch erkannt	Variiert	Die meisten Benutzer

Resource-Pool-Architektur

Die Resource Pools sind hierarchisch aufgebaut:

- **Ebene 1 – control_plane** (`max_concurrent_jobs = 16`): Begrenzt Gesamtzahl aktiver Tile-Jobs
- **Ebene 2 – executor** (`max_concurrent_tasks = 128`): Begrenzt alle Tasks im Executor
- **Ebene 3 – Resource Pools** (aufgeteilt nach Ressourcentyp):
 - *Network Pool* (128 Verbindungen): HTTP/TLS Downloads
 - *CPU Pool* (10 Operationen): Tile-Assembly + DDS-Encode
 - *Disk Pool* (64 Operationen): Cache Lese-/Schreibzugriffe

Performance-Empfehlung: 16 HT-Cores mit paralleler CPU-Last

Ein System mit 16 logischen CPUs (z.B. 8 physische Kerne + 8 Hyperthreading) erzeugt folgende **Defaults**:

Einstellung	Default-Wert	Formel
<code>generation.threads</code>	16	$= \text{num_cpus}$
<code>executor.cpu_concurrent</code>	20	$\max(16 \times 1.25, 16 + 2) = 20$
<code>control_plane.max_concurrent_jobs</code>	32	$16 \times 2 = 32$

Problem: Konkurrenz mit X-Plane und anderen Programmen

Mit den Defaults beansprucht XEarthLayer **alle 16 logischen Kerne** gleichzeitig. Wenn parallel CPU-intensive Programme laufen (X-Plane Rendering, Streaming-Software, andere Simulationen), führt das zu:

- **Thread-Überbelegung**: Mehr aktive Threads als verfügbare Kerne, OS muss ständig Context-Switches durchführen
- **Hyperthreading-Limitierung**: HT-Kerne teilen sich physische Ressourcen (ALU, Cache). Zwei CPU-intensive Threads auf demselben physischen Kern erreichen zusammen nur ca. 120–130% der Leistung eines einzelnen Threads, nicht 200%
- **X-Plane-Stuttering**: X-Plane ist stark Mainthread-gebunden. Wenn XEarthLayer alle Kerne sättigt, steigt die Frametime

Empfohlene Konfiguration bei paralleler CPU-Last

Faustregel: Auf die Hälfte der *physischen* Kerne beschränken (= 1/4 der HT-Cores).

```
[generation]
threads = 4                ; Statt 16: nur 4 Worker-Threads

[executor]
cpu_concurrent = 4         ; Statt 20: max 4 gleichzeitige Encode-Ops

[control_plane]
max_concurrent_jobs = 8    ; Statt 32: max 8 parallele Tile-Jobs
```

Abstufungen je nach Szenario

Szenario	threads	cpu_concurrent	max_concurrent_jobs	Bemerkung
XEL allein (Default)	16	20	32	Maximale Tile-Geschwindigkeit
XEL + X-Plane	6–8	6–8	12–16	Guter Kompromiss
XEL + X-Plane + Stream- ing/Recording	4	4	8	Konservativ, stotterfrei
XEL im Hintergrund (minimale Last)	2	2	4	Nur bei Bedarf Tiles generieren

Hinweise

- **DDS-Encoding ist die CPU-intensivste Operation** (BC1/BC3-Kompression von 4096x4096 Tiles, ca. 0.2s pro Tile). `cpu_concurrent` ist daher der wirkungsvollste Hebel.
- **Tile-Geschwindigkeit sinkt proportional:** Halbierung der Threads halbiert ungefähr den Durchsatz. Das Prefetch-System passt sich automatisch an (Calibration erkennt niedrigeren Durchsatz und wählt ggf. *Opportunistic* statt *Aggressive* Modus).
- **Netzwerk ist oft der Flaschenhals**, nicht die CPU. Bei langsamer Internetverbindung bringt eine Reduktion der CPU-Threads kaum Performanceverlust, da die Threads ohnehin auf Downloads warten.

Konfigurationsdatei: `~/.xearthlayer/config.ini`